

Testkonzept für das Open-Source-Projekt „lazygit„

Belegarbeit im Studiengang Informatik (B.Sc.)

**Grundlagen des Softwaretestens
Wintersemester 2025/26**

Dozent: Prof. Dr. Matthias Längrich

Hochschule Zittau/Görlitz
Fakultät Elektrotechnik und Informatik

**Verfasser: Konrad Miosga
Matrikelnummer: 1056710
E-Mail-Adresse: konrad.miosga@stud.hszg.de**

Datum der Abgabe: 25.01.2026

Inhaltsverzeichnis

1. Einleitung	1
2. Theoretische Grundlagen	2
3. Analyse der vorhandenen Teststrategie	5
3.1. Unit-Tests	5
3.2. Table-Driven Tests	6
3.3. Integrationstests	7
4. Entwurf eigener Testfälle	8
4.1. Identifikation von Testlücken	8
4.2. Analyse	8
4.3. Testfalldesign	11
4.4. Implementierung	14
4.5. Testergebnisse und Coverage-Verbesserung	15
5. Fazit	16
Anhang	17
Abbildungen	17
Quellen	23

1. Einleitung

Moderne Softwareprojekte zeichnen sich durch hohe Komplexität, kurze Entwicklungszyklen und kontinuierliche Erweiterung aus. Insbesondere Open-Source-Projekte werden häufig von verteilten Entwicklergruppen weiterentwickelt und unterliegen einer stetigen Veränderung des Funktionsumfangs. Unter diesen Bedingungen ist eine verlässliche und wartbare Testinfrastruktur entscheidend, um bestehende Funktionalität abzusichern und Regressionen frühzeitig zu erkennen.

Das Open-Source-Projekt lazygit ist eine in Go entwickelte Terminal-basierte Benutzeroberfläche für Git, die komplexe Versionskontrolloperationen über eine tastaturgesteuerte Oberfläche zugänglich macht. Aufgrund seiner weiten Verbreitung und aktiven Entwicklung eignet sich lazygit als praxisnahes Fallbeispiel für die Untersuchung von Teststrategien in realen Softwareprojekten.

Ziel dieser Belegarbeit ist es, das bestehende Testkonzept von lazygit zu analysieren, dessen Stärken und Schwächen herauszuarbeiten und durch die Implementierung zusätzlicher Testfälle gezielt zu erweitern. Der Schwerpunkt liegt dabei auf der funktionalen Absicherung zentraler Git-Operationen sowie auf der strukturellen Abdeckung relevanter Codepfade.

Die Arbeit ist wie folgt aufgebaut: Zunächst werden die notwendigen theoretischen Grundlagen des Softwaretestens erläutert. Anschließend wird die Architektur und vorhandene Testinfrastruktur von lazygit betrachtet. Darauf aufbauend werden Anforderungen, Use Cases und Testbedingungen für ausgewählte Funktionalitäten abgeleitet und mithilfe geeigneter Testentwurfstechniken in konkrete Testfälle überführt. Den Abschluss bilden eine Zusammenfassung der Ergebnisse sowie ein Ausblick auf mögliche Weiterentwicklungen.

2. Theoretische Grundlagen

Software-Testing ist ein zentraler Bestandteil der Qualitätssicherung in der Softwareentwicklung. Das oft zitierte Statement von Edsger W. Dijkstra bringt dabei eine grundlegende Eigenschaft des Testens prägnant auf den Punkt: Tests eignen sich hervorragend zum Aufdecken von Fehlern, sind jedoch ungeeignet, um die vollständige Fehlerfreiheit eines Programms nachzuweisen [1].

Diese Einschränkung ergibt sich aus der Natur des Software-Testings als Stichprobenverfahren. Da Programme in der Regel eine sehr große Menge möglicher Eingaben besitzen, kann im Rahmen von Tests nur eine endliche Teilmenge davon überprüft werden. Ein formaler Beweis der Korrektheit allein durch Tests ist daher prinzipiell nicht möglich.

Vor diesem Hintergrund kommt der systematischen und zielgerichteten Auswahl repräsentativer Testfälle eine zentrale Bedeutung zu. Ziel des Testens ist es nicht, wie schon eingangs erwähnt, Fehlerfreiheit zu garantieren, sondern mit begrenztem Aufwand eine möglichst hohe Wahrscheinlichkeit zur Entdeckung relevanter Defekte zu erreichen.

Der Testprozess lässt sich grundsätzlich in zwei zentrale Bereiche gliedern: die statische Analyse und das dynamische Testen [2, S. 4]. Die statische Analyse untersucht Softwareartefakte ohne deren Ausführung und zielt darauf ab, potenzielle Fehler, Regelverletzungen oder Qualitätsmängel frühzeitig im Entwicklungsprozess zu identifizieren [2, S. 43–45]. Zu den typischen Verfahren zählen Code-Reviews, der Einsatz von Lintern sowie statische Datenfluss- und Kontrollflussanalysen.

Demgegenüber steht das dynamische Testen, bei dem die Software mit konkreten Eingabedaten ausgeführt wird. Dadurch lassen sich insbesondere solche Fehler aufdecken, die erst zur Laufzeit auftreten, etwa Speicherlecks, Race Conditions oder fehlerhaftes Laufzeitverhalten. Dynamische Tests ermöglichen somit eine Überprüfung des tatsächlichen Systemverhaltens unter realistischen oder gezielt konstruierten Bedingungen [2, S. 39–43]. Der Schwerpunkt dieser Arbeit liegt auf dem dynamischen Testen und den zugehörigen Testverfahren.

Dynamisches Testen lässt sich in zwei grundlegende Herangehensweisen unterteilen: Black-Box- und White-Box-Tests.

Beim Black-Box-Test werden Testfälle ausschließlich aus der externen Spezifikation eines Systems abgeleitet, ohne Kenntnisse seiner inneren Implementierung. Man behandelt die Software als „schwarze Box“, und konzentriert sich darauf, ob die funktionalen und nicht-funktionalen Anforderungen erfüllt sind. Zu den klassischen Black-Box-Verfahren gehören

die Äquivalenzklassenbildung und die Grenzwertanalyse. Bei der Äquivalenzklassenbildung werden Eingabedaten in Gruppen zusammengefasst, von denen man annimmt, dass sie vom System gleichartig verarbeitet werden. Statt alle Mitglieder einer Klasse zu testen, wählt man einen repräsentativen Vertreter aus. Die Grenzwertanalyse ergänzt dieses Vorgehen, indem sie Testfälle gezielt an den Rändern dieser Äquivalenzklassen platziert, da dort erfahrungsgemäß häufig Fehler auftreten [3, S. 115].

Der White-Box-Test erfordert hingegen detaillierte Kenntnisse der internen Programmstruktur. Testfälle werden hierbei so entworfen, dass bestimmte Strukturelemente des Codes, wie Anweisungen, Verzweigungen oder Pfade, gezielt durchlaufen werden. Das Ziel ist es, eine definierte Code-Abdeckung (Coverage) zu erreichen und die korrekte Implementierung der internen Logik zu verifizieren [3, S. 128-129].

In der Testpraxis werden beide Ansätze selten isoliert betrachtet. Vielmehr werden sie in einem systematischen Testentwurfsprozess kombiniert, um sowohl die funktionale Korrektheit (Black-Box-Sicht) als auch die strukturelle Robustheit (White-Box-Sicht) sicherzustellen. Ein solcher Prozess, wie er auch im praktischen Teil dieser Arbeit zur Anwendung kommt, beginnt oft mit einer Black-Box-Sicht, bei der aus funktionalen Anforderungen grobe Testideen oder Use Cases abgeleitet werden. Daraufhin folgt die White-Box-Analyse der konkreten Implementierung, um entscheidungsrelevante Codepfade zu identifizieren. Basierend auf dieser Code-Logik können dann White-Box-orientierte Äquivalenzklassen gebildet werden; so hat ein Parameter, der eine `if`-Bedingung steuert, beispielsweise die beiden Äquivalenzklassen `true` und `false`, die für eine vollständige Zweigabdeckung getestet werden müssen. Um schließlich die Interaktion verschiedener Parameter effizient zu überprüfen, kommen kombinatorische Testverfahren wie das paarweise Testen zum Einsatz [4, S. 192]. Anstatt alle denkbaren Parameterkombinationen zu testen, was zu einer Testfallexplosion führen würde, wählt dieser Ansatz Testfälle so aus, dass jede mögliche Kombination von zwei Parametern mindestens einmal abgedeckt ist. Dies stellt einen pragmatischen Kompromiss zwischen Aufwand und Fehlerfindungsrate dar, da die meisten Softwarefehler durch die Interaktion von wenigen Parametern entstehen [5].

Durch diese Kombination wird sichergestellt, dass die Tests nicht nur relevante Anwenderszenarien abdecken, sondern auch alle logischen Verzweigungen im Code systematisch validieren.

Innerhalb der White-Box-Tests lassen sich verschiedene, aufeinander aufbauende Testverfahren unterscheiden. Unit-Tests überprüfen die kleinsten testbaren Einheiten eines

Programms in Isolation. Abhängige Komponenten werden dabei typischerweise durch Mocks oder Stubs ersetzt, was schnelle, reproduzierbare und deterministische Tests ermöglicht [2, S. 371–372]. Eine häufig eingesetzte Ausprägung sind sogenannte Table-Driven Tests, bei denen mehrere Testfälle in Form von Datentabellen definiert und von einem generischen Testcode iterativ ausgeführt werden [6]. Dieses Vorgehen reduziert Redundanz und verbessert die Wartbarkeit der Tests.

Aufbauend auf den Unit-Tests folgen Integrationstests, die das Zusammenwirken mehrerer bereits getesteter Module überprüfen. Ziel ist es insbesondere, Fehler an Schnittstellen sowie Probleme im Zusammenspiel der Komponenten aufzudecken [2, S. 372–376]. Im Gegensatz zu Unit-Tests kommen hierbei überwiegend reale Komponenten statt isolierender Mocks zum Einsatz, wodurch eine höhere Aussagekraft hinsichtlich der Gesamtfunktionalität des Systems erreicht wird.

3. Analyse der vorhandenen Teststrategie

Lazygit testet auf zwei Ebenen: Unit-Tests und Integrationstests. Die Unit-Tests prüfen einzelne Funktionen isoliert und laufen sehr schnell, während die Integrationstests die gesamte Anwendung mit echter Benutzerinteraktion testen. Dabei kommen sogenannte Table-Driven Tests zum Einsatz, eine Go-typische Methode, um viele ähnliche Testfälle kompakt zu schreiben.

3.1. Unit-Tests

Die Unit-Tests in lazygit arbeiten mit Mocks, das heißt sie führen keine echten Git-Befehle aus, sondern simulieren diese. Das Kernstück ist der `FakeCmdObjRunner`, der vorgibt, Git-Befehle auszuführen, tatsächlich aber nur aufzeichnet, welche Befehle aufgerufen wurden, und vordefinierte Antworten zurückgibt. So kann man Tests schreiben, die schnell, zuverlässig und unabhängig vom tatsächlichen Git-Zustand sind.

Ein typischer Unit-Test in lazygit sieht so aus:

```
1 func TestBranchNewBranch(t *testing.T) {
2     runner := oscommands.NewFakeRunner(t).
3     ExpectGitArgs([]string{"checkout", "-b", "test", "refs/heads/master"}, "", nil)
4     instance := buildBranchCommands(commonDeps{runner: runner})
5
6     assert.NoError(t, instance.New("test", "refs/heads/master"))
7     runner.CheckForMissingCalls()
8 }
```

Man sagt dem Fake-Runner vorher, welchen Git-Befehl man erwartet, führt dann die zu testende Funktion aus, und prüft am Ende, ob wirklich der erwartete Befehl aufgerufen wurde. Das ist einfach und funktioniert gut für Komponenten, die hauptsächlich Git-Befehle zusammenbauen und ausführen.

3.2. Table-Driven Tests

Table-Driven Tests sind in Go sehr verbreitet. Die Idee ist, dass man mehrere Testfälle in einer Liste definiert und dann in einer Schleife durchläuft. Das spart Code-Duplikation und macht es einfach, neue Testfälle hinzuzufügen. In Lazygit wird das konsequent eingesetzt, besonders wenn man verschiedene Eingaben und Fehlerfälle durchprobieren will.

Ein Beispiel aus `branch_test.go` zeigt, wie das aussieht:

```
1  scenarios := []scenario{
2      {
3          "Can't retrieve pushable count",
4          oscommands.NewFakeRunner(t).
5              ExpectGitArgs([]string{"rev-list", "@{u}..HEAD", "--count"}, "",
6                  errors.New("error")),
7          "?", "?",
8      },
9      {
10         "Retrieve pullable and pushable count",
11         oscommands.NewFakeRunner(t).
12             ExpectGitArgs([]string{"rev-list", "@{u}..HEAD", "--count"}, "1\n", nil).
13             ExpectGitArgs([]string{"rev-list", "HEAD..@{u}", "--count"}, "2\n", nil),
14         "1", "2",
15     },
16 }
17 for _, s := range scenarios {
18     t.Run(s.testName, func(t *testing.T) {
19         instance := buildBranchCommands(commonDeps{runner: s.runner})
20         pushables, pullables := instance.GetCommitDifferences("HEAD", "@{u}")
21         assert.EqualValues(t, s.expectedPushables, pushables)
22         assert.EqualValues(t, s.expectedPullables, pullables)
23         s.runner.CheckForMissingCalls()
24     })
25 }
```

Jedes Szenario hat einen Namen, eine Mock-Konfiguration und erwartete Ergebnisse. Die Schleife führt dann für jeden Fall den gleichen Test aus. Wenn ein Test fehlschlägt, sieht man sofort am Namen, welches Szenario das Problem hat.

3.3. Integrationstests

Die Integrationstests sind das Herzstück der Teststrategie. Sie testen nicht einzelne Funktionen, sondern komplette User-Workflows. Dafür wurde ein eigenes Test-Framework entwickelt, mit der man UI-Interaktionen beschreiben kann.

Ein Test für einen Rebase sieht zum Beispiel so aus:

```
1  var Rebase = NewIntegrationTest(NewIntegrationTestArgs{ go
2      Description: "Rebase onto another branch, deal with the conflicts.",
3      SetupRepo: func(shell *Shell) {
4          shared.MergeConflictsSetup(shell)
5      },
6      Run: func(t *TestDriver, keys config.KeybindingConfig) {
7          t.Views().Commits().TopLines(
8              Contains("first change"),
9              Contains("original"),
10         )
11
12         t.Views().Branches().
13             Focus().
14             Lines(
15                 Contains("first-change-branch"),
16                 Contains("second-change-branch"),
17             ).
18             SelectNextItem().
19             Press(keys.Branches.RebaseBranch)
20
21         t.ExpectPopup().Menu().
22             Title(Equals("Rebase 'first-change-branch'")).
23             Select(Contains("Simple rebase")).
24             Confirm()
25
26         t.Common().AcknowledgeConflicts()
27     },
28 }
```

Der Test läuft in drei Schritten ab: Zuerst wird ein echtes Git-Repository mit dem Shell-Helper vorbereitet. Dann simuliert der TestDriver Benutzeraktionen wie `Focus()`, `SelectNextItem()` oder `Press()` – das entspricht den Tastendrücken, die ein echter Nutzer machen würde. Zwischendurch wird immer wieder geprüft, ob die UI das Erwartete anzeigt, zum Beispiel mit `Contains()` oder `Equals()`. Man kann den Test lesen wie eine Beschreibung dessen, was ein Nutzer tut.

Technisch basiert das Framework auf einer Architektur spezialisierter Driver-Komponenten. Der `ViewDriver` ermöglicht Interaktionen mit Listen-Views wie `Branches`, `Commits` und `Files`,

während der `MenuDriver` die Navigation in Popup-Menüs steuert. Der `PromptDriver` behandelt Texteingaben, der `ConfirmationDriver` das Bestätigen oder Ablehnen von Dialogen und der `AlertDriver` die Validierung von Fehlermeldungen. Diese Abstraktion entkoppelt die Testlogik von der konkreten UI-Implementation, was Refactorings erheblich erleichtert. Ändert sich die Struktur der Benutzeroberfläche, müssen lediglich die Driver-Implementierungen angepasst werden, während die hunderten von Tests unverändert bleiben können.

4. Entwurf eigener Testfälle

Der praktische Teil dieser Arbeit bestand darin, Testlücken im Lazygit-Projekt zu identifizieren und durch eigene Testfälle zu schließen. Dabei wurden die zuvor analysierten Teststrategien angewendet und die Coverage messbar verbessert.

4.1. Identifikation von Testlücken

Die Analyse der Coverage-Reports in Kombination mit manuellen Code-Reviews identifizierte zwei signifikante Testlücken. Die Tag-Kommandos in `pkg/commands/git_commands/tag.go` enthielten acht Funktionen ohne jegliche Testabdeckung. Tags sind in Git zentral für die Versionsverwaltung, weshalb fehlerhafte Implementierungen zu versehentlich überschriebenen oder falsch gesetzten Tags führen können.

1	<code>/lazygit/pkg/commands/git_commands/tag.go:14:</code>	<code>NewTagCommands</code>	0.0%	<code>bash</code>
2	<code>/lazygit/pkg/commands/git_commands/tag.go:20:</code>	<code>CreateLightweightObj</code>	0.0%	
3	<code>/lazygit/pkg/commands/git_commands/tag.go:30:</code>	<code>CreateAnnotatedObj</code>	0.0%	
4	<code>/lazygit/pkg/commands/git_commands/tag.go:40:</code>	<code>HasTag</code>	0.0%	
5	<code>/lazygit/pkg/commands/git_commands/tag.go:49:</code>	<code>LocalDelete</code>	0.0%	
6	<code>/lazygit/pkg/commands/git_commands/tag.go:56:</code>	<code>Push</code>	0.0%	
7	<code>/lazygit/pkg/commands/git_commands/tag.go:71:</code>	<code>ShowAnnotationInfo</code>	0.0%	
8	<code>/lazygit/pkg/commands/git_commands/tag.go:80:</code>	<code>IsTagAnnotated</code>	0.0%	

4.2. Analyse

Anforderungsanalyse

Die Tag-Verwaltung in lazygit bildet zentrale Funktionalitäten der Git-Versionskontrolle ab. Als fachliche Grundlage dienen die offizielle Referenzdokumentation des Git-Befehls `git tag` [7] sowie das Pro Git-Handbuch [8]. Aus diesen Quellen lassen sich die funktionalen Fähigkeiten ableiten, die eine Tag-Verwaltung mindestens unterstützen muss.

Auf dieser Basis ergeben sich die in [Tabelle 1](#) zusammengefassten funktionalen Anforderungen:

ID	Anforderung
FA-01	Erstellung von Lightweight Tags (einfache Commit-Pointer)
FA-02	Erstellung von Annotated Tags mit Metadaten (Autor, Datum, Message)
FA-03	Erstellung von Tags auf beliebigen Commits (nicht nur HEAD)
FA-04	Überschreiben existierender Tags mittels Force-Option
FA-05	Lokales Löschen von Tags
FA-06	Unterscheidung zwischen Annotated und Lightweight Tags

Tabelle 1: Funktionale Anforderungen

Diese Anforderungen beschreiben was das System leisten muss, unabhängig davon, wie Benutzer diese Funktionen konkret auslösen.

Use-Case-Analyse

Aus den identifizierten funktionalen Anforderungen wurden typische Nutzungsszenarien abgeleitet, die reale Arbeitsabläufe von Entwicklern abbilden. Diese Use Cases bündeln jeweils mehrere Anforderungen und bilden die Grundlage für das spätere Testdesign.

Die resultierenden Use Cases sind in [Tabelle 2](#) dargestellt.

ID	Beschreibung
UC-01	Release-Version taggen: Commit mit Versionskennzeichnung versehen
UC-02	Fehlerhaften Tag lokal korrigieren
UC-03	Tag-Informationen anzeigen und Typ unterscheiden
UC-04	Bestehenden Tag verschieben (Rolling Tags aktualisieren)

Tabelle 2: Use Cases

Der Use Case *UC-01: Release-Version taggen* stellt den primären Workflow dar. Der Benutzer navigiert zu einem Commit, vergibt einen Tagnamen und entscheidet optional, ob ein Lightweight- oder ein Annotated Tag erstellt werden soll. Existiert der Tag bereits, ist eine explizite Bestätigung zum Überschreiben erforderlich. Dieser Use Case deckt insbesondere FA-01 bis FA-04 ab.

Der Use Case *UC-02: Fehlerhaften Tag lokal korrigieren* beschreibt das Szenario, in dem ein Entwickler einen bereits existierenden Tag lokal entfernt oder überschreibt, bevor dieser in ein Remote-Repository übertragen wird. Ziel ist es, inkonsistente oder fehlerhafte Tag-

Zustände frühzeitig zu bereinigen. Dieser Use Case adressiert primär FA-04 (Überschreiben existierender Tags) und FA-05 (Löschen von Tags).

Der Use Case *UC-03: Tag-Informationen anzeigen und Typ unterscheiden* umfasst das Anzeigen vorhandener Tags sowie die Unterscheidung zwischen Lightweight- und Annotated Tags. Diese Funktion ist notwendig, um korrekt interpretieren zu können, ob zu einem Tag zusätzliche Metadaten wie Autor oder Nachricht existieren. UC-03 validiert insbesondere FA-06.

Der Use Case *UC-04: Bestehenden Tag verschieben* adressiert Szenarien wie das Korrigieren falsch gesetzter Tags oder das Aktualisieren von Rolling Tags (z. B. latest, stable). Hierbei wird die Force-Option verwendet, wodurch zusätzliche Sicherheitsprüfungen notwendig sind.

Testbedingungen

Aus den beschriebenen Use Cases wurden konkrete Testbedingungen abgeleitet. Diese beschreiben unter welchen Voraussetzungen ein Testfall ausgeführt wird und welches Verhalten erwartet wird. Jede Testbedingung lässt sich mindestens einem Use Case zuordnen.

Die resultierenden Testbedingungen sind in [Tabelle 3](#) aufgeführt.

ID	Bedingung	Erwartetes Verhalten	Use Case
TB-01	Lightweight Tag ohne Ref	<code>git tag <name></code> wird ausgeführt	UC-01
TB-02	Tag auf spezifischem Commit	<code>git tag <name> <ref></code>	UC-01, UC-04
TB-03	Force-Flag gesetzt	<code>--force</code> ist enthalten	UC-02, UC-04
TB-04	Force + spezifischer Commit	Kombination beider Parameter	UC-04
TB-05	Annotated Tag mit Message	<code>-m <message></code> Parameter	UC-01
TB-06	Tag-Typ erkennen	<code>git cat-file -t</code> Ausgabe wird ausgewertet	UC-03
TB-07	Tag löschen	<code>git tag -d <name></code>	UC-02

Tabelle 3: Testbedingungen

Diese Testbedingungen bilden die direkte Grundlage für die spätere Definition konkreter Testfälle und stellen die Nachvollziehbarkeit von Anforderungen bis zu Tests sicher.

4.3. Testfalldesign

Ziel des Testfalldesigns ist es, aus den definierten Testbedingungen konkrete, ausführbare Testfälle abzuleiten. Da die Tag-Funktionalität in lazygit Git-Befehle programmgesteuert über Hilfsfunktionen wie `NewGitCmd` und `ArgIf` zusammensetzt, ergibt sich die maßgebliche Testlogik direkt aus den im Code enthaltenen Verzweigungen.

Die Funktion `ArgIf(condition, arg)` fügt ein Argument ausschließlich dann zur Befehlsliste hinzu, wenn die übergebene Bedingung erfüllt ist. Jede solche Bedingung erzeugt alternative Code-Pfade und muss daher durch geeignete Testfälle abgedeckt werden.

Code-Analyse für `CreateLightweightObj`

Die Funktion `CreateLightweightObj(tagName string, ref string, force bool)` enthält folgende entscheidungsrelevante Anweisungen:

```
1 NewGitCmd("tag").
2   ArgIf(force, "--force").
3   Arg("--", tagName).
4   ArgIf(len(ref) > 0, ref).
```

go

Hieraus ergeben sich zwei logische Entscheidungen:

- Ob das Flag `--force` hinzugefügt wird (`force == true`)
- Ob ein Referenz-Commit übergeben wird (`len(ref) > 0`)

Der Parameter `tagName` durchläuft keine bedingte Logik und wird stets unverändert an Git weitergereicht.

Äquivalenzklassenbildung

Auf Basis der identifizierten Verzweigungen werden die Parameter in Äquivalenzklassen eingeteilt.

Parameter	Äquivalenzklasse	Bedeutung / Effekt
<code>tagName</code>	beliebiger String	Wird unverändert an Git übergeben
<code>ref</code>	leer ("")	Kein Ref-Argument wird ergänzt
<code>ref</code>	nicht-leer (z. B. "abc123")	Ref-Argument wird ergänzt
<code>force</code>	false	--force wird nicht ergänzt
<code>force</code>	true	--force wird ergänzt

Tabelle 4: Äquivalenzklassen

Diese Klasseneinteilung bildet exakt die im Code vorhandenen Entscheidungspunkte ab.

Kombinatorische Testabdeckung

Aus den zuvor definierten Äquivalenzklassen ergeben sich für die Funktion `CreateLightweightObj` folgende Variationsräume:

- 1 Klasse für `tagName`
- 2 Klassen für `ref`
- 2 Klassen für `force`

Damit existieren insgesamt:

$1 \times 2 \times 2 = 4$ mögliche Parameterkombinationen

Da die Anzahl an Kombinationen überschaubar ist und es somit nicht zu einer Testfall-Explosion kommt, werden sämtliche Kombinationen explizit getestet. Auf diese Weise wird sichergestellt, dass jeder im Code vorhandene Verzweigungspfad mindestens einmal durchlaufen wird.

Test	ref	force	Erwartetes Kommando
TF-01	leer	false	<code>git tag -- v1.0.0</code>
TF-02	nicht-leer	false	<code>git tag -- v1.0.0 abc123</code>
TF-03	leer	true	<code>git tag --force -- v1.0.0</code>
TF-04	nicht-leer	true	<code>git tag --force -- v1.0.0 def456</code>

Tabelle 5: Testfälle für `CreateLightweightObj`

TF-04 stellt den kritischsten Testfall dar, da hier beide optionalen Argumente gleichzeitig aktiv sind und zusätzlich die korrekte Reihenfolge der erzeugten Git-Argumente überprüft wird.

Diese vier Testfälle decken die Testbedingungen TB-01 bis TB-04 vollständig ab.

Die Testfälle TF-05 bis TF-08 werden analog zu den zuvor beschriebenen Fällen abgeleitet. Die Funktion `CreateAnnotatedObj` besitzt zusätzlich den Parameter `msg`, der sich wie `tagName` verhält und keine eigene Verzweigungslogik enthält. Der Parameter wird stets unverändert an Git weitergereicht.

Daher ergeben sich auch für `CreateAnnotatedObj` lediglich vier relevante Parameterkombinationen, die durch TF-05 bis TF-08 abgedeckt werden.

Grenzwertanalyse

Die Funktion `IsTagAnnotated` parst Git-Output und muss robuste String-Verarbeitung gewährleisten. Getestet werden:

- Exakte Übereinstimmung: `"tag\n"` (Annotated) vs. `"commit\n"` (Lightweight)
- Whitespace-Toleranz: `" tag \n"` (mit führenden/nachfolgenden Leerzeichen)
- Edge Case: Leerer String (implizit durch `strings.TrimSpace` abgedeckt)

Table-Driven Testing

Das Implementierungspattern folgt dem Projekt-Standard. Jeder Test definiert eine Szenario-Struktur mit Eingabeparametern und erwarteten Git-Argumenten, die in einer Schleife ausgeführt werden. Dies ermöglicht kompakte, wartbare Tests mit klarer Trennung von Testdaten und Testlogik.

Testfallübersicht

Testfall	Funktion	Zweck	Entwurfstechnik
TF-01	<code>CreateLightweightObj</code>	Lightweight Tag auf HEAD	Kombinatorik
TF-02	<code>CreateLightweightObj</code>	Lightweight Tag auf spezifischem Commit	Kombinatorik
TF-03	<code>CreateLightweightObj</code>	Lightweight Tag mit Force	Kombinatorik
TF-04	<code>CreateLightweightObj</code>	Force + spezifischer Commit	Kombinatorik
TF-05	<code>CreateAnnotatedObj</code>	Annotated Tag auf HEAD	Analog zu TF-01–TF-04
TF-06	<code>CreateAnnotatedObj</code>	Annotated Tag auf spezifischem Commit	Analog zu TF-01–TF-04
TF-07	<code>CreateAnnotatedObj</code>	Annotated Tag mit Force	Analog zu TF-01–TF-04
TF-08	<code>CreateAnnotatedObj</code>	Annotated Tag mit Force + Commit	Analog zu TF-01–TF-04
TF-09	<code>IsTagAnnotated</code>	Annotated Tag erkennen	Grenzwertanalyse
TF-10	<code>IsTagAnnotated</code>	Lightweight Tag erkennen	Grenzwertanalyse
TF-11	<code>IsTagAnnotated</code>	Whitespace tolerant parsen	Grenzwertanalyse
TF-12	<code>LocalDelete</code>	Lokales Löschen eines Tags	Direkter Funktionsaufruf

Tabelle 6: Übersicht aller Testfälle

Diese Testfälle validieren alle Testbedingungen aus [Tabelle 3](#) und decken somit alle funktionalen Anforderungen aus [Tabelle 1](#) ab.

4.4. Implementierung

Die Tag-Tests wurden in `tag_test.go` implementiert und folgen strikt den Projekt-Konventionen. Jeder Testfall definiert eine Szenario-Struktur mit Eingabeparametern und erwarteten Git-Argumenten. Beispielhaft sei hier das implementierte Szenario für TF-01 dargestellt.

```
1 //... go
2 func TestTagCommands_CreateLightweightObj(t *testing.T) {
3     type scenario struct {
4         testName      string
5         tagName         string
6         ref             string
7         force          bool
8         expectedCmdArgs []string
9     }
10
11     scenarios := []scenario{
12         {
13             testName:      "create simple lightweight tag on HEAD",
14             tagName:         "v1.0.0",
15             ref:             "",
16             force:          false,
17             expectedCmdArgs: []string{"git", "tag", "--", "v1.0.0"},
18         }
19     },
20     //...
21     for _, s := range scenarios {
22         t.Run(s.testName, func(t *testing.T) {
23             runner := oscommands.NewFakeRunner(t)
24             gitCommon := buildGitCommon(commonDeps{runner: runner})
25             tagCommands := NewTagCommands(gitCommon)
26
27             cmdObj := tagCommands.CreateLightweightObj(s.tagName, s.ref, s.force)
28
29             assert.Equal(t, s.expectedCmdArgs, cmdObj.Args())
30         })
31     }
32 }
```

Der Test verwendet `oscommands.NewFakeRunner`, um eine isolierte Testumgebung aufzubauen, ohne externe Prozesse auszuführen. Im vorliegenden Fall wird jedoch kein Git Aufruf simuliert, sondern ausschließlich geprüft, ob `CreateLightweightObj` die erwartete Argumentliste für den Git Befehl korrekt zusammensetzt. Die Validierung erfolgt durch den Vergleich von `cmdObj.Args()` mit den im Szenario definierten `expectedCmdArgs`.

Für Tests, die tatsächlich einen Git Aufruf auslösen, wird der Fake Runner zusätzlich mit erwarteten Argumenten konfiguriert und anschließend mit `CheckForMissingCalls()` verifiziert. Dies ist beispielsweise bei `IsTagAnnotated` und `LocalDelete` der Fall.

Die vollständige Implementierung ist in [Abbildung 1](#) ersichtlich.

4.5. Testergebnisse und Coverage-Verbesserung

Die Ausführung der implementierten Tests zeigt, dass alle Testfälle erfolgreich durchlaufen. Die zugehörigen Testergebnisse sind in [Abbildung 2](#) dargestellt.

Die Coverage-Analyse zeigt signifikante Verbesserungen für `tag.go`, wo fünf von acht Funktionen auf 100% Coverage gebracht wurden:

1	/lazygit/pkg/commands/git_commands/tag.go:14:	NewTagCommands	100.0%	bash
2	/lazygit/pkg/commands/git_commands/tag.go:20:	CreateLightweightObj	100.0%	
3	/lazygit/pkg/commands/git_commands/tag.go:30:	CreateAnnotatedObj	100.0%	
4	/lazygit/pkg/commands/git_commands/tag.go:40:	HasTag	0.0%	
5	/lazygit/pkg/commands/git_commands/tag.go:49:	LocalDelete	100.0%	
6	/lazygit/pkg/commands/git_commands/tag.go:56:	Push	0.0%	
7	/lazygit/pkg/commands/git_commands/tag.go:71:	ShowAnnotationInfo	0.0%	
8	/lazygit/pkg/commands/git_commands/tag.go:80:	IsTagAnnotated	100.0%	

Die Funktion `NewTagCommands` wurde in allen Tests implizit mitgetestet, da sie in allen Testfällen Verwendung findet. Die drei verbleibenden Funktionen (`HasTag`, `Push`, `ShowAnnotationInfo`) wurden in dieser Arbeit nicht getestet. Bei `Push` handelt es sich um eine Remote-Operation die Netzwerk-Kommunikation erfordert. `HasTag` und `ShowAnnotationInfo` sind unterkomplex und wurden nicht priorisiert.

Die Gesamt-Coverage von `tag.go` stieg von 0% auf 62.5%.

5. Fazit

Im Rahmen dieser Arbeit wurde das Testkonzept des Projekts lazygit anhand ausgewählter Funktionalitäten untersucht und gezielt erweitert. Die Analyse der bestehenden Testinfrastruktur zeigte, dass das Projekt bereits über eine solide Basis automatisierter Tests verfügt, insbesondere durch den konsequenten Einsatz von table-driven Tests und durch die Entkopplung von externen Abhängigkeiten mittels Fake-Runnern.

Gleichzeitig konnten durch Coverage-Analysen und manuelle Code-Reviews Bereiche identifiziert werden, in denen zentrale Logikpfade bislang nicht oder nur unzureichend abgesichert waren. Für diese Bereiche wurden funktionale Anforderungen, Use Cases und Testbedingungen systematisch hergeleitet und in konkrete Testfälle überführt. Die implementierten Tests verbessern insbesondere die Absicherung der Git-Befehlskonstruktion in der Tag-Verwaltung und erhöhen die Nachvollziehbarkeit des Testentwurfs durch klare Ableitungsbeziehungen.

Die Ergebnisse zeigen, dass sich mit überschaubarem Aufwand und methodischem Vorgehen die Testabdeckung und Aussagekraft einer bestehenden Test-Suite deutlich steigern lassen. Dabei hat sich insbesondere die Kombination aus Code-Analyse und klassischen Testentwurfstechniken als praktikabel erwiesen.

Anhang

Abbildungen

```
1 //tag_test.go go
2 package git_commands
3
4 import (
5     "testing"
6
7     "github.com/jesseduffield/lazygit/pkg/commands/oscommands"
8     "github.com/stretchr/testify/assert"
9 )
10
11 func TestTagCommands_CreateLightweightObj(t *testing.T) {
12     type scenario struct {
13         testName      string
14         tagName        string
15         ref            string
16         force          bool
17         expectedCmdArgs []string
18     }
19
20     scenarios := []scenario{
21         {
22             //TF-01
23             testName:      "create simple lightweight tag on HEAD",
24             tagName:         "v1.0.0",
25             ref:             "",
26             force:           false,
27             expectedCmdArgs: []string{"git", "tag", "--", "v1.0.0"},
28         },
29         {
30             //TF-02
31             testName:      "create lightweight tag on specific commit",
32             tagName:         "v1.0.0",
33             ref:             "abc123",
34             force:           false,
35             expectedCmdArgs: []string{"git", "tag", "--", "v1.0.0", "abc123"},
36         },
37         {
38             //TF-03
39             testName:      "create lightweight tag with force flag",
40             tagName:         "v1.0.0",
41             ref:             "",
42             force:           true,
43             expectedCmdArgs: []string{"git", "tag", "--force", "--", "v1.0.0"},
44         },
45     }
```

```

46 //TF-04
47 testName:      "create forced lightweight tag on specific commit",
48 tagName:       "v1.0.0",
49 ref:           "def456",
50 force:         true,
51 expectedCmdArgs: []string{"git", "tag", "--force", "--", "v1.0.0", "def456"},
52 },
53 }
54
55 for _, s := range scenarios {
56     t.Run(s.testName, func(t *testing.T) {
57         runner := oscommands.NewFakeRunner(t)
58         gitCommon := buildGitCommon(commonDeps{runner: runner})
59         tagCommands := NewTagCommands(gitCommon)
60
61         cmdObj := tagCommands.CreateLightweightObj(s.tagName, s.ref, s.force)
62
63         assert.Equal(t, s.expectedCmdArgs, cmdObj.Args())
64     })
65 }
66 }
67
68 func TestTagCommands_CreateAnnotatedObj(t *testing.T) {
69     type scenario struct {
70         testName      string
71         tagName        string
72         ref            string
73         msg            string
74         force          bool
75         expectedCmdArgs []string
76     }
77
78     scenarios := []scenario{
79         {
80             //TF-05
81             testName:      "create annotated tag on HEAD",
82             tagName:         "v1.0.0",
83             ref:              "",
84             msg:              "Release version 1.0.0",
85             force:            false,
86             expectedCmdArgs: []string{"git", "tag", "v1.0.0", "-m", "Release version 1.0.0"},
87         },
88         {
89             //TF-06
90             testName:      "create annotated tag on specific commit",
91             tagName:         "v1.0.0",
92             ref:             "abc123",
93             msg:             "Major release",
94             force:            false,

```

```

95     expectedCmdArgs: []string{"git", "tag", "v1.0.0", "abc123", "-m", "Major
96         },
97     {
98         //TF-07
99         testName:      "create forced annotated tag",
100        tagName:        "v1.0.0",
101        ref:             "",
102        msg:             "Latest stable",
103        force:           true,
104        expectedCmdArgs: []string{"git", "tag", "v1.0.0", "--force", "-m", "Latest
105            stable"},
106    },
107    //TF-08
108    testName:      "create forced annotated tag on specific commit",
109    tagName:        "v1.0.0",
110    ref:            "xyz789",
111    msg:            "Beta version",
112    force:          true,
113    expectedCmdArgs: []string{"git", "tag", "v1.0.0", "--force", "xyz789", "-m",
114        "Beta version"},
115 }
116
117 for _, s := range scenarios {
118     t.Run(s.testName, func(t *testing.T) {
119         runner := oscommands.NewFakeRunner(t)
120         gitCommon := buildGitCommon(commonDeps{runner: runner})
121         tagCommands := NewTagCommands(gitCommon)
122
123         cmdObj := tagCommands.CreateAnnotatedObj(s.tagName, s.ref, s.msg, s.force)
124
125         assert.Equal(t, s.expectedCmdArgs, cmdObj.Args())
126     })
127 }
128 }
129
130 func TestTagCommands_IsTagAnnotated(t *testing.T) {
131     type scenario struct {
132         testName      string
133         tagName        string
134         gitOutput      string
135         gitError       error
136         expectedResult bool
137         expectedError  error
138     }
139
140     scenarios := []scenario{
141         {
142             //TF-09

```

```

143     testName:      "tag is annotated",
144     tagName:        "v1.0.0",
145     gitOutput:      "tag\n",
146     gitError:       nil,
147     expectedResult: true,
148     expectedError:  nil,
149 },
150 {
151     //TF-10
152     testName:      "tag is lightweight",
153     tagName:        "v1.0.0",
154     gitOutput:      "commit\n",
155     gitError:       nil,
156     expectedResult: false,
157     expectedError:  nil,
158 },
159 {
160     //TF-11
161     testName:      "tag with extra whitespace",
162     tagName:        "v1.0.0",
163     gitOutput:      " tag \n",
164     gitError:       nil,
165     expectedResult: true,
166     expectedError:  nil,
167 },
168 }
169
170 for _, s := range scenarios {
171     t.Run(s.testName, func(t *testing.T) {
172         runner := oscommands.NewFakeRunner(t).
173             ExpectGitArgs([]string{"cat-file", "-t", "refs/tags/" + s.tagName},
174                 s.gitOutput, s.gitError)
175
176         gitCommon := buildGitCommon(commonDeps{runner: runner})
177         tagCommands := NewTagCommands(gitCommon)
178
179         result, err := tagCommands.IsTagAnnotated(s.tagName)
180
181         assert.Equal(t, s.expectedResult, result)
182         if s.expectedError != nil {
183             assert.Error(t, err)
184         } else {
185             assert.NoError(t, err)
186         }
187         runner.CheckForMissingCalls()
188     })
189 }
190
191 //TF-12

```

```

192 func TestTagCommands_LocalDelete(t *testing.T) {
193     runner := oscommands.NewFakeRunner(t).
194         ExpectGitArgs([]string{"tag", "-d", "v1.0.0"}, "", nil)
195
196     gitCommon := buildGitCommon(commonDeps{runner: runner})
197     tagCommands := NewTagCommands(gitCommon)
198
199     err := tagCommands.LocalDelete("v1.0.0")
200
201     assert.NoError(t, err)
202     runner.CheckForMissingCalls()
203 }

```

Abbildung 1: tag_test.go

```

1  # go test -v ./pkg/commands/git_commands -run TestTag bash
2  === RUN   TestTagCommands_CreateLightweightObj
3  === RUN   TestTagCommands_CreateLightweightObj/create_simple_lightweight_tag_on_HEAD
4  === RUN   TestTagCommands_CreateLightweightObj/create_lightweight_tag_on_specific_commit
5  === RUN   TestTagCommands_CreateLightweightObj/create_lightweight_tag_with_force_flag
6  === RUN   TestTagCommands_CreateLightweightObj/create_forced_lightweight_tag_on_specific_commit
7  --- PASS: TestTagCommands_CreateLightweightObj (0.00s)
8  --- PASS: TestTagCommands_CreateLightweightObj/create_simple_lightweight_tag_on_HEAD (0.00s)
9  --- PASS: TestTagCommands_CreateLightweightObj/create_lightweight_tag_on_specific_commit (0.00s)
10 --- PASS: TestTagCommands_CreateLightweightObj/create_lightweight_tag_with_force_flag (0.00s)
11 --- PASS: TestTagCommands_CreateLightweightObj/create_forced_lightweight_tag_on_specific_commit (0.00s)
12 === RUN   TestTagCommands_CreateAnnotatedObj
13 === RUN   TestTagCommands_CreateAnnotatedObj/create_annotated_tag_on_HEAD
14 === RUN   TestTagCommands_CreateAnnotatedObj/create_annotated_tag_on_specific_commit
15 === RUN   TestTagCommands_CreateAnnotatedObj/create_forced_annotated_tag
16 === RUN   TestTagCommands_CreateAnnotatedObj/create_forced_annotated_tag_on_specific_commit
17 --- PASS: TestTagCommands_CreateAnnotatedObj (0.00s)
18 --- PASS: TestTagCommands_CreateAnnotatedObj/create_annotated_tag_on_HEAD (0.00s)
19 --- PASS: TestTagCommands_CreateAnnotatedObj/create_annotated_tag_on_specific_commit (0.00s)
20 --- PASS: TestTagCommands_CreateAnnotatedObj/create_forced_annotated_tag (0.00s)
21 --- PASS: TestTagCommands_CreateAnnotatedObj/create_forced_annotated_tag_on_specific_commit (0.00s)
22 === RUN   TestTagCommands_LocalDelete
23 --- PASS: TestTagCommands_LocalDelete (0.00s)
24 === RUN   TestTagCommands_IsTagAnnotated
25 === RUN   TestTagCommands_IsTagAnnotated/tag_is_annotated
26 === RUN   TestTagCommands_IsTagAnnotated/tag_is_lightweight
27 === RUN   TestTagCommands_IsTagAnnotated/tag_with_extra_whitespace
28 --- PASS: TestTagCommands_IsTagAnnotated (0.00s)
29 --- PASS: TestTagCommands_IsTagAnnotated/tag_is_annotated (0.00s)
30 --- PASS: TestTagCommands_IsTagAnnotated/tag_is_lightweight (0.00s)
31 --- PASS: TestTagCommands_IsTagAnnotated/tag_with_extra_whitespace (0.00s)
32 PASS
33 ok      github.com/jesseduffield/lazygit/pkg/commands/git_commands  0.004s

```

Abbildung 2: Testergebnisse für tag_test.go

Quellen

- [1] E. W. Dijkstra, „The Humble Programmer“, *ACM Turing Award Lectures*. Association for Computing Machinery, New York, NY, USA, 2007. doi: 10.1145/1283920.1283927.
- [2] P. Liggesmeyer, *Software-Qualität: Testen, Analysieren und Verifizieren von Software*, 2. Auflage. Spektrum Akademischer Verlag, 2009.
- [3] A. Spillner und T. Linz, *Basiswissen Softwaretest: Aus- und Weiterbildung zum Certified Tester – Foundation Level nach ISTQB-Standard*. dpunkt.verlag, 2003.
- [4] D. W. Hoffmann, *Software-Qualität*, 2. Auflage. Springer Vieweg, 2013.
- [5] D. R. Kuhn, R. N. Kacker, und Y. Lei, „Practical Combinatorial Testing“, Gaithersburg, MD, technical report NIST Special Publication 800-142, 2010. doi: 10.6028/NIST.SP.800-142.
- [6] The Go Authors, „Table Driven Tests“. Zugegriffen: 15. Januar 2026. [Online]. Verfügbar unter: <https://github.com/golang/go/wiki/TableDrivenTests>
- [7] „Git - git-tag Documentation“. Zugegriffen: 15. Januar 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-tag>
- [8] S. Chacon und B. Straub, „Git Basics: Tagging“. Zugegriffen: 15. Januar 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Basics-Tagging>